



- * Since Queue is an interface, we cannot create its object directly. Instead we use classes that implement it.
- * Queue → basic first-in first-out (FIFO) structure
- * Deque → Double Ended Queue (insert/remove from both ends)

LinkedList implementation of Queue:

1. Creating Queue object -

Queue q = new Queue(); ✗

Queue<Integer> q = new LinkedList<>(); ✓

Queue<Integer> → reference type, LinkedList<> → actual implementation

2. Queue methods -

Method	Work	Empty queue behaviour
add()	insert element	-
offer()	insert element safely	-
remove()	remove front element	throws exception
poll()	remove safely	returns null
element()	view front element	throws exception
peek()	view safely	returns null

Traversing Queue

Three ways to traverse:

1. For-each loop

```
for(Integer x : q)
{
    System.out.println(x);
}
```

2. Iterator

```
Iterator<Integer> it = q.iterator();
while(it.hasNext())
{
    System.out.println(it.next());
}
```

3. While loop

```
while(!q.isEmpty())
{
    System.out.println(q.poll());
}
```

Sample Program

```
import java.util.*;
public class QueueExample
{
    public static void main(String[] args)
    {
        Queue<Integer> q = new LinkedList<>();
        q.add(10);
        q.add(20);
        q.add(30);
        q.add(40);

        System.out.println("Queue: " + q);
        System.out.println("Front element: " + q.peek());
        q.remove();
        System.out.println("After removal: " + q);
    }
}
```

Input/Output:

Queue : 10 20 30 40
front rear

Front element: 10

Queue : 20 30 40
front rear

Frame

Time Complexity:

Operation	Time Complexity
add() / offer()	O(1)
remove() / poll()	O(1)
peek() / element()	O(1)
traversal	O(n)

In a queue implemented using LinkedList, insertion happens at the rear and deletion happens at the front, so both operations take constant time.

Space Complexity: O(n)

The queue stores n elements, so the memory required grows linearly with the number of elements